



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LAB 08 INSTRUCTIONS

For

Reconstruct an application dependency chain

Course Code: C1123

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 5.1

Lab 08 — Reconstruct an application dependency chain

Course: Wireshark Network Analysis Masterclass (C1123)

Topic: 08 — TCP/IP Communications and Resolution

Learning outcome: LO2 — Navigate, filter, summarize and time network exchanges.

Time: about 30 minutes

Version: v5.1 · 5 October 2026

Scenario

A user says "the portal is down". Before blaming the web server, you rebuild every dependency the browser needed: address resolution, name resolution, connection and request.

Goal

ARP resolves a local next-hop MAC; DNS resolves a name to an address.

What you will produce

A completed dependency map with frame ranges.

Before you start

- Wireshark 4.6 or later, with the TShark command-line tools on PATH.
- Python 3 for the verification and export scripts (Windows: use `py -3` in place of `python3`).
- Open this lab folder in a terminal; every file the lab needs is inside it.
- Use only the supplied synthetic captures — do not capture on a network you are not authorised to monitor.

Files for this lab

File	What it is for
data/branch-office.pcap	Synthetic branch-office capture used by the lab
assets/scenario.md	The help-desk ticket that sets the scenario
assets/checks.json	Filters and frame counts the fixture must satisfy
assets/expected-evidence.png	The packet list your filter should produce
scripts/verify.py	Checks the capture facts with TShark
scripts/export_evidence.py	Exports this lab's evidence rows to outputs/evidence.csv
outputs/findings.md	Findings sheet you complete
assets/dependency-map.md	Dependency chain template
assets/topology.md	Fictional branch topology

Lab at a glance

1. Locate the ARP exchange and DNS lookup
2. Draw the TCP/HTTP exchange in Flow Graph
3. Fill in the dependency map
4. Contrast a local and a routed next hop

Step-by-step

1. Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
python3 scripts/verify.py
```

2. Open `data/branch-office.pcap`. Locate the ARP exchange and DNS transaction 101 for `portal.example.test`. Record the server address `192.0.2.20`.

3. Apply `tcp.stream == 0` and open Statistics > Flow Graph with displayed packets selected. Identify SYN, SYN/ACK, ACK, request and response.

4. Fill `assets/dependency-map.md` with the ARP, DNS, TCP and HTTP frame ranges. Distinguish Ethernet next hop from IP destination.

5. Contrast a same-subnet server with a routed server using `assets/topology.md`: a routed destination needs the gateway MAC rather than ARP for the remote host. Record what this trace can and cannot demonstrate.

6. Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in `outputs/findings.md`.

```
python3 scripts/export_evidence.py
```

Expected evidence

With the lab filter applied, your packet list should match the frames below (produced by TShark from this lab's own capture).

dns.qry.name == "portal.example.test"					
No.	Time	Source	Destination	Protocol	Info
3	0.020000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0065 A portal.example.test
4	0.040000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0065 A portal.example.t...
branch-office.pcap Displayed: 2					

Lab 08 expected evidence

Test it

The supplied `dns.qry.name == "portal.example.test"` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Troubleshooting

- TShark not found: install Wireshark CLI tools and add the installation folder to PATH; on Windows use the Wireshark install directory.
- A filter returns zero: clear other filters, use the specified capture, and check the expression is in the display toolbar.

- TLS remains opaque: select the matching lab-tls.keys file by absolute path, reload, and remove a key-log preference from another lab.

Try it with TShark

The same evidence from the command line — run it from this lab folder:

```
tshark -n -r data/branch-office.pcap -Y "arp || dns.id == 0x0065 ||  
tcp.stream == 0"
```

Challenge

Develop and explain an alternative filter.

Reflection

Which second observation point would strengthen your conclusion?

Extension (optional)

Ethernet and ARP lab — trace the dependency chain for a page you load on an authorised network. Source: J.F. Kurose and K.W. Ross, Wireshark Labs (gaia.cs.umass.edu/kurose_ross/wireshark.php).

Reset

Clear all display filters and return to the C1123-Analyst or Default profile. If you loaded a TLS key log, remove it from Preferences > Protocols > TLS. The supplied captures never need regenerating for the core lab.

Note: The same steps appear in the Learner Guide. The slides show only the scenario and a summary.